BACKEND ARCHITECTURE BLUEPRINT

Flash Sale Engine & API Rate Limiting Gateway

Production-Grade System Design

Spring Boot 3.x · MySQL 8.x · Redis 7.x · ReactJS

Prepared: 21 June 2026

Architecture Level: Staff / Principal Engineer

Table of Contents

01 Architecture Overview

System Purpose

This system provides two tightly coupled backend capabilities: an API Rate Limiting Gateway that protects every endpoint from abuse, and a Flash Sale Engine that handles high-concurrency purchase events with atomic inventory management and zero oversell guarantees. Redis is the single source of truth for all real-time state; MySQL is the system of record.

Core Components

Component	Responsibility	Technology
Rate Limiter Filter	Intercepts every request, enforces per-user/per-IP request quotas via Redis. Returns 429 on breach.	Spring OncePerRequestFilter + Redis
Idempotency Filter	Deduplicates purchase requests using client-supplied X-Idempotency-Key. Prevents double charges on retries.	Spring Filter + Redis SETNX
Flash Sale Service	Manages sale events, atomically decrements inventory, enforces per-user purchase limits.	Spring Service + Redis Lua
Purchase Order Service	Writes confirmed orders to MySQL asynchronously after Redis inventory lock succeeds.	Spring Service + Redis List queue
SSE Inventory Stream	Broadcasts real-time inventory changes to connected React clients.	Spring SseEmitter + Redis Pub/Sub
Sale Admin API	Creates sale events, loads inventory into Redis, adjusts rate limit configs.	Spring REST Controller
MySQL	Stores orders, sale events, users, products, rate limit configs. System of record.	MySQL 8.x + Spring Data JPA
Redis	Holds all real-time state: rate limit counters, idempotency keys, inventory counts, pub/sub channels.	Redis 7.x (Lettuce client)

Request Flow — Purchase Endpoint

The following layers execute in sequence for every POST /api/v1/sales/{saleId}/purchase call:

1. Client sends POST with JWT bearer token and X-Idempotency-Key header.
2. **Rate Limiter Filter:** Extracts userId from JWT. Checks Redis sorted set for sliding window. If limit exceeded → 429 Too Many Requests with Retry-After header. Adds < 5ms overhead.
3. **Idempotency Filter:** Checks Redis for idempotency key. If DONE → return cached response. If PROCESSING → return 202. If absent → SET NX with PROCESSING state.
4. **Auth Check:** JWT signature verified. User must have ROLE_USER or ROLE_VIP.
5. **Sale Window Check:** Confirm current timestamp is within sale startTime/endTime from MySQL (cached in Redis with 30s TTL).
6. **Per-User Limit Check:** HGET userPurchases:{saleId}:{userId} — check units already purchased vs maxPerUser.
7. **Atomic Inventory Decrement:** Lua script executes: check inventory > 0, DECR inventory:{saleId}, HINCRBY userPurchases — all atomic. Returns 409 if sold out.
8. **Async Order Write:** LPUSH orders:queue with order payload. Background BRPOP worker writes to MySQL orders table.
9. **Inventory Broadcast:** PUBLISH stock:updates:{saleId} with new count. SSE endpoint relays to React clients.
10. **Idempotency Finalise:** Store final response body in Redis idem:{key} with 24h TTL, mark DONE.

Event Flow

Event	Trigger	Consumers
sale.started	Admin sets startTime, cron fires	Inventory loader job → SETEX inventory:{id} count
purchase.completed	Lua script succeeds	Order writer (Redis List) + SSE broadcaster (Pub/Sub)
sale.ended	Cron detects endTime passed	Reconciliation job: Redis inventory → MySQL sale_event.final_count
rate.limit.breached	Sliding window counter exceeds limit	Logging + metrics increment (Micrometer counter)
inventory.zero	DECR returns 0	Pub/Sub → SSE 'SOLD OUT' broadcast

Architecture Decision: Why Redis Over DB for Real-Time State

Key Insight

At 5,000 concurrent purchase requests, a MySQL UPDATE SET inventory = inventory - 1 WHERE inventory > 0 creates a row-level lock queue that serialises requests. Measured latency: 200-800ms per request under load. Redis DECR is a single-threaded atomic O(1) operation: latency stays under 1ms regardless of concurrency. The systems have complementary roles, not competing ones.

Technology Stack

02 Technology Stack Selection

Backend Framework — Spring Boot 3.x

Attribute	Detail
Why chosen	Built-in filter chain (OncePerRequestFilter) maps directly to rate limiter + idempotency layers. Spring Data JPA + Hibernate handles MySQL ORM. Spring's SseEmitter is production-grade for SSE. Actuator provides metrics endpoint for Prometheus.
Redis client	Lettuce (not Jedis). Lettuce is non-blocking, uses a single shared connection, and natively supports Redis pipelined commands and reactive streams. Jedis is thread-per-connection — adds connection overhead under high concurrency.
Alternatives considered	Quarkus (smaller image, faster startup — worth it if deploying to GraalVM native), Micronaut (less ecosystem), plain Spring MVC (no benefit over Boot).
Why not alternatives	Spring Boot's ecosystem maturity and community support outweigh startup-time differences for this use case. Boot 3.x with virtual threads (Java 21) closes the reactive performance gap.

Database — MySQL 8.x

Attribute	Detail
Why chosen	System of record for orders, users, products, sale events. Strong ACID guarantees for financial writes. MySQL 8 window functions support analytics queries. JSON column type enables flexible product metadata storage.
Alternatives	PostgreSQL (slightly better JSON support, better concurrency for writes), CockroachDB (distributed, overkill at this scale), DynamoDB (no relational guarantees, wrong for order data).
Why not PostgreSQL	MySQL 8 is the dominant choice at Indian product companies (Flipkart, Meesho, Razorpay). For interview relevance and ecosystem familiarity, MySQL is the pragmatic pick. Switch to PostgreSQL at scale if write contention on orders table becomes a bottleneck.

Cache & Real-Time State — Redis 7.x

Attribute	Detail
Why chosen	Rate limiting (sorted sets, INCR + EXPIRE), idempotency (SETNX with TTL), inventory (atomic DECR, Lua scripts), session storage, Pub/Sub for SSE fan-out, and List-based job queue — all native Redis primitives. No other technology covers all these use cases with sub-millisecond latency.
Redis 7 specifically	Introduces Redis Functions (replaces EVAL-based Lua in production), improved ACL controls, and multi-part AOF for better persistence. LMPOP for non-blocking list consumption.
Alternatives	Memcached (no data structures beyond key-value, no Pub/Sub, no persistence), Hazelcast (JVM-only distributed cache, heavier operational overhead), in-process cache (not shared across instances).
Why not alternatives	Memcached cannot implement sliding window rate limiting, idempotency state machines, or Pub/Sub. Every rate limiter algorithm in this system requires Redis sorted sets or Lua atomicity.
Persistence config	AOF (appendonly yes) + RDB snapshot. Inventory data must survive Redis restart — RDB alone risks losing last N seconds of purchases. AOF with everysec fsync is the safe default.

ORM — Spring Data JPA + Hibernate

Attribute	Detail
Why chosen	Repository pattern matches the layered architecture. Named queries prevent SQL injection by default. Schema migration via Flyway gives version-controlled DB changes.
Alternatives	MyBatis (explicit SQL, better for complex queries), JOOQ (type-safe SQL, excellent for reporting), plain JDBC (no boilerplate benefits).

Attribute	Detail
Tradeoff	Hibernate's N+1 problem is a real risk on order-listing queries. Use @EntityGraph or fetch joins explicitly. NEVER use FetchType.EAGER.

Observability Stack

Tool	Purpose	Why Chosen
Micrometer + Prometheus	Metrics collection — request counts, rate limit breach rate, inventory decrements/sec, Redis latency histograms	Micrometer is Spring Boot's native metrics facade. Zero configuration for Actuator endpoint.
Grafana	Metrics dashboards and alerting	Standard open-source pairing with Prometheus. Pre-built Spring Boot dashboard templates available.
Logback + JSON appender	Structured logging to stdout (Docker collects)	JSON logs are parseable by log aggregators. Correlation IDs added via MDC filter.
OpenTelemetry (future)	Distributed tracing across Filter → Service → Redis → MySQL	Spring Boot 3.x has native OTel support. Add when moving to multi-instance deployment.

Load Testing — k6

Attribute	Detail
Why k6	JavaScript test scripts, built-in thresholds (fail if p95 > 100ms), virtual users simulate concurrency, native CSV output for analysis. Runs in Docker — no install required.
Alternative	Apache JMeter (GUI-based, heavier, good for beginners), Gatling (Scala DSL, excellent reports). k6 is the modern standard for API load testing.

Attribute	Detail
Key test	500 virtual users all hitting POST /purchase simultaneously at sale start. Verify: zero duplicate orders, exact inventory count in MySQL matches expected, no 500 errors.

Development Phases

03 Production Development Phases

For a 12–15 hour implementation, follow Phases 1–3 completely. Phases 4–7 are production-readiness extensions for post-learning iteration.

Phase 1 — MVP Foundation (Hours 1–3)

Goal
Working Spring Boot project with Redis connected, fixed-window rate limiter active, and a single product endpoint protected.

Item	Detail	Deliverable
Docker Compose	Redis 7 + MySQL 8 containers with health checks and volume mounts	docker-compose.yml
Spring Boot skeleton	Maven project: spring-boot-starter-web, spring-boot-starter-data-jpa, spring-data-redis (Lettuce), spring-boot-starter-actuator, flyway-core	pom.xml
Fixed window rate limiter	RateLimiterFilter.java implementing INCR + EXPIRE. 10 req/min per userId.	RateLimiterFilter.java
Redis config	RedisConfig.java with Lettuce connection factory, StringRedisTemplate, RedisTemplate<String, Object>	RedisConfig.java
Flyway V1 migration	users, products tables with seed data	V1__initial_schema.sql
Product API	GET /api/v1/products (protected by rate limiter)	ProductController.java

Phase 2 — Core Redis Logic (Hours 4–9)

Goal
All three rate limiting algorithms, idempotency layer, and flash sale with atomic inventory decrement working correctly under concurrent load.

Item	Detail	Deliverable
Sliding window rate limiter	ZADD + ZREMRANGEBYSCORE + ZCARD in RateLimiterService. Switch via application.yml ratelimit.algorithm=SLIDING_WINDOW	RateLimiterService.java
Token bucket (Lua)	Lua script: load tokens, calculate refill since last call, decrement. Atomic via EVALSHA.	token_bucket.lua + LuaScriptConfig.java
Idempotency filter	IdempotencyFilter.java: SETNX idem:{key} PROCESSING, wrap response, store in Redis, mark DONE	IdempotencyFilter.java + IdempotencyStore.java
Sale event admin API	POST /api/v1/admin/sales — create SaleEvent in MySQL. POST /api/v1/admin/sales/{id}/activate — SETEX inventory:{id} count EX {duration}	SaleAdminController.java
Flash sale Lua script	Atomic: check inventory > 0, DECR, HINCRBY userPurchases:{saleId}:{userId}. Returns new count or -1 for sold out.	flash_sale_purchase.lua
Purchase endpoint	POST /api/v1/sales/{saleId}/purchase — orchestrates Lua script, pushes to Redis List queue	PurchaseController.java + FlashSaleService.java
Order writer job	@Scheduled BRPOP orders:queue — pop order, write to MySQL orders table with retry on failure	OrderPersistenceJob.java
Flyway V2 migration	sale_events, orders, rate_limit_configs tables	V2__flash_sale_schema.sql

Phase 3 — Real-Time & Testing (Hours 10–13)

Goal
SSE inventory stream live in React, k6 concurrency test passing with zero oversell, all Redis patterns verified.

Item	Detail	Deliverable
Redis Pub/Sub publisher	On every inventory change: RedisTemplate.convertAndSend("stock:updates:"+saleId, count)	InventoryPublisher.java
SSE endpoint	GET /api/v1/sales/{saleId}/inventory/stream — SseEmitter subscribing to Redis channel	InventoryStreamController.java
React SSE consumer	EventSource("/api/v1/sales/{id}/inventory/stream") updating stock counter	InventoryCounter.jsx
k6 load test	500 VUs, 30s duration, all hitting purchase endpoint. Assert: p95 < 200ms, 0 orders > inventory	load-test.js
Redis CLI verification	KEYS inventory:* | HGETALL | LLEN orders:queue — manual verification commands documented	TESTING.md
Algorithm comparison	Benchmark fixed vs sliding vs token bucket at 1000 RPS. Document latency and memory differences	benchmark-results.md

Phase 4 — Observability (Post-MVP)

Item	Detail	Deliverable
Prometheus metrics	Custom counters: rate_limit_breaches_total, inventory_decrements_total, idempotency_hits_total	MetricsConfig.java
Grafana dashboard	Spring Boot + Redis latency + inventory drain rate panels	grafana-dashboard.json
Structured logging	Logback JSON appender, MDC correlation ID (X-Request-ID) propagated through all layers	logback-spring.xml
Health checks	Custom HealthIndicator for Redis connectivity, MySQL, and active sale count	SaleHealthIndicator.java

Phase 5 — Performance (Post-MVP)

Item	Detail	Deliverable
Redis pipeline	Batch ZADD + EXPIRE calls in sliding window using pipelined() for multi-instance scenarios	RateLimiterService refactor
MySQL connection pool	HikariCP tuning: maximumPoolSize=20, connectionTimeout=3000, validationTimeout=1000	application.yml tuning
Redis connection pool	Lettuce pool: maxTotal=8, minIdle=2 — prevents connection starvation under burst	RedisConfig.java tuning
Sale event caching	Cache SaleEvent in Redis Hash for 30s TTL — avoids DB hit on every purchase validation	SaleEventCache.java

Project Structure

04 Backend Project Structure

Architecture Philosophy: Feature-Based Layers

This project uses a hybrid approach: top-level organisation by technical layer (controller, service, repository, filter) but grouped within feature packages (ratelimit, flashsale, order). This prevents the anti-pattern of 20 files in a single controller/ directory while still making the layer boundaries clear for new developers.

Full Directory Tree

```
flash-sale-system/
├── src/main/java/com/company/flashsale/
│   ├── FlashSaleApplication.java          ← Spring Boot entry point
│   │
│   ├── config/                            ← All @Configuration classes
│   │   ├── RedisConfig.java               ← Lettuce factory, templates, Lua scripts
│   │   ├── SecurityConfig.java            ← JWT filter chain, CORS
│   │   └── MetricsConfig.java             ← Custom Micrometer counters
│   │
│   ├── filter/                            ← Servlet filters (execute before
controllers)
│   │   ├── RateLimiterFilter.java         ← Algorithm selection + Redis check
│   │   ├── IdempotencyFilter.java         ← Idempotency key lifecycle
│   │   └── CorrelationIdFilter.java       ← MDC X-Request-ID propagation
│   │
│   ├── ratelimit/                         ← Rate limiting feature
│   │   ├── RateLimiterService.java        ← Algorithm implementations
│   │   ├── RateLimitConfig.java           ← Entity: per-endpoint config
│   │   ├── RateLimitConfigRepository.java
│   │   ├── RateLimitResult.java           ← Value object: allowed/blocked/remaining
│   │   └── algorithm/
│   │       ├── FixedWindowStrategy.java
│   │       ├── SlidingWindowStrategy.java
│   │       └── TokenBucketStrategy.java   ← Lua script execution
│   │
│   ├── flashsale/                         ← Flash sale feature
│   │   ├── SaleEvent.java                 ← Entity
```

```
│   │   ├── SaleEventRepository.java
│   │   ├── FlashSaleService.java          ← Business logic + Lua orchestration
│   │   ├── InventoryPublisher.java        ← Redis Pub/Sub publish
│   │   ├── InventoryStreamController.java ← SSE /inventory/stream endpoint
│   │   ├── SaleAdminController.java       ← Admin endpoints
│   │   └── dto/
│   │       ├── CreateSaleRequest.java
│   │       ├── PurchaseRequest.java
│   │       └── PurchaseResponse.java
│   │
│   ├── order/                             ← Order persistence feature
│   │   ├── Order.java                     ← Entity
│   │   ├── OrderRepository.java
│   │   ├── OrderPersistenceJob.java       ← BRPOP consumer, writes to MySQL
│   │   └── OrderService.java
│   │
│   ├── product/                           ← Product catalogue
│   │   ├── Product.java
│   │   ├── ProductRepository.java
│   │   └── ProductController.java
│   │
│   ├── user/                              ← User management
│   │   ├── User.java
│   │   ├── UserRepository.java
│   │   └── UserService.java
│   │
│   ├── auth/                              ← Authentication
│   │   ├── AuthController.java            ← /login, /refresh
│   │   ├── JwtService.java                ← Token generation + validation
│   │   └── JwtAuthFilter.java
│   │
│   ├── idempotency/                       ← Idempotency support
│   │   ├── IdempotencyStore.java          ← Redis operations for idem keys
│   │   └── IdempotencyState.java          ← Enum: PROCESSING / DONE
│   │
│   └── common/                            ← Shared across all features
```

```
│       ├── exception/
│       │   ├── RateLimitExceededException.java
│       │   ├── SoldOutException.java
│       │   └── GlobalExceptionHandler.java ← @RestControllerAdvice
│       ├── response/
│       │   └── ApiResponse.java            ← Standard response envelope
│       └── util/
│           └── RedisKeyBuilder.java        ← Centralised key naming constants
│
├── src/main/resources/
│   ├── application.yml                     ← Base config
│   ├── application-local.yml               ← Local Docker overrides
│   ├── application-prod.yml                ← Production overrides
│   └── scripts/lua/                        ← Lua scripts loaded at startup
│       ├── token_bucket.lua
│       ├── sliding_window.lua
│       └── flash_sale_purchase.lua
│
├── src/main/resources/db/migration/        ← Flyway migrations
│   ├── V1__initial_schema.sql
│   ├── V2__flash_sale_schema.sql
│   └── V3__rate_limit_config.sql
│
├── src/test/
│   ├── java/com/company/flashsale/
│   │   ├── ratelimit/
│   │   │   └── RateLimiterServiceTest.java ← Unit test all 3 algorithms
│   │   ├── flashsale/
│   │   │   └── FlashSaleServiceTest.java   ← Concurrency test with CountDownLatch
│   │   └── integration/
│   │       └── PurchaseFlowIntegrationTest.java ← @SpringBootTest + Testcontainers
│   └── resources/
│       └── application-test.yml
│
├── load-tests/
│   └── k6/
```

```
│       ├── flash-sale-load.js          ← 500 VU purchase load test
│       └── rate-limit-breach.js        ← Rate limit algorithm comparison
│
├── docker/
│   ├── docker-compose.yml              ← Redis + MySQL + App
│   ├── Dockerfile                      ← Multi-stage: build + runtime
│   └── redis.conf                      ← AOF + maxmemory config
│
├── docs/
│   └── REDIS_KEY_DESIGN.md             ← Every key, TTL, and why
│
└── pom.xml
```

What Never Goes Where

Location	Purpose	Never Place Here
config/	Spring @Configuration classes only	Business logic, entity classes, service code
filter/	Servlet filters executing before controller chain	Service calls that need transaction management — delegate to service layer
common/exception/	Exceptions + GlobalExceptionHandler	Feature-specific business logic
scripts/lua/	Lua scripts as classpath resources, loaded at startup via ClassPathResource	Inline Lua strings in Java code — maintenance nightmare
db/migration/	Flyway SQL migrations only — numbered, never edited after deploy	Seed data for production (use separate seed migration flagged with profile)

Database Design

05 Database Design — MySQL 8.x

V1 — Initial Schema

```
-- V1__initial_schema.sql

CREATE TABLE users (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    uuid        VARCHAR(36)  NOT NULL UNIQUE,          -- exposed in APIs
    email       VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role        ENUM('USER','VIP','ADMIN') NOT NULL DEFAULT 'USER',
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    deleted_at  DATETIME(3) NULL,                      -- soft delete
    INDEX idx_email (email),
    INDEX idx_uuid (uuid),
    INDEX idx_deleted_at (deleted_at)                  -- filter active users
efficiently
);

CREATE TABLE products (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    uuid        VARCHAR(36)  NOT NULL UNIQUE,
    name        VARCHAR(255) NOT NULL,
    description TEXT,
    base_price  DECIMAL(12,2) NOT NULL,
    metadata    JSON,                                   -- flexible product
attributes
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    deleted_at  DATETIME(3) NULL,
    INDEX idx_uuid (uuid)
);
```

V2 — Flash Sale Schema

```
-- V2__flash_sale_schema.sql

CREATE TABLE sale_events (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    uuid            VARCHAR(36)  NOT NULL UNIQUE,
    product_id      BIGINT UNSIGNED NOT NULL,
    sale_price      DECIMAL(12,2) NOT NULL,
    total_inventory INT UNSIGNED  NOT NULL,
    final_count     INT UNSIGNED  NULL,        -- populated by reconciliation job after sale ends
    max_per_user    TINYINT UNSIGNED NOT NULL DEFAULT 1,
    start_time      DATETIME(3) NOT NULL,
    end_time        DATETIME(3) NOT NULL,
    status          ENUM('DRAFT','ACTIVE','ENDED','CANCELLED') NOT NULL DEFAULT 'DRAFT',
    created_by      BIGINT UNSIGNED NOT NULL,  -- admin userId
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
    INDEX idx_status_start (status, start_time),   -- cron job uses this
    INDEX idx_uuid (uuid)
);

CREATE TABLE orders (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    uuid            VARCHAR(36)  NOT NULL UNIQUE,      -- returned to client immediately
    user_id         BIGINT UNSIGNED NOT NULL,
    sale_event_id   BIGINT UNSIGNED NOT NULL,
    product_id      BIGINT UNSIGNED NOT NULL,
    quantity        TINYINT UNSIGNED NOT NULL DEFAULT 1,
    unit_price      DECIMAL(12,2) NOT NULL,
    total_price     DECIMAL(12,2) NOT NULL,
```

```
    status          ENUM('PENDING','CONFIRMED','CANCELLED') NOT NULL DEFAULT
'CONFIRMED',
    idempotency_key VARCHAR(36)  NOT NULL UNIQUE,      -- prevents duplicate
persistence
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    FOREIGN KEY (user_id)       REFERENCES users(id),
    FOREIGN KEY (sale_event_id) REFERENCES sale_events(id),
    FOREIGN KEY (product_id)    REFERENCES products(id),
    INDEX idx_user_sale (user_id, sale_event_id),     -- per-user order history
    INDEX idx_sale_event (sale_event_id),             -- admin: all orders for a
sale
    INDEX idx_idempotency (idempotency_key),
    INDEX idx_created_at (created_at)                 -- time-series queries
);
```

V3 — Rate Limit Config Schema

```
-- V3__rate_limit_config.sql

CREATE TABLE rate_limit_configs (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    endpoint_pattern VARCHAR(255) NOT NULL,    -- e.g. POST:/api/v1/sales/*/purchase
    algorithm       ENUM('FIXED_WINDOW','SLIDING_WINDOW','TOKEN_BUCKET') NOT NULL,
    requests_limit  INT UNSIGNED NOT NULL,     -- max requests per window
    window_seconds  INT UNSIGNED NOT NULL,     -- window duration in seconds
    burst_capacity  INT UNSIGNED NULL,         -- token bucket only: max burst
    refill_rate     DECIMAL(8,2) NULL,         -- token bucket only: tokens/second
    applies_to_role ENUM('USER','VIP','ADMIN','IP') NOT NULL DEFAULT 'USER',
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    UNIQUE KEY uq_endpoint_role (endpoint_pattern, applies_to_role),
```

```
    INDEX idx_active (is_active)
);

-- VIP override example: 100 purchases/min vs 10 for normal users
INSERT INTO rate_limit_configs VALUES
  (NULL, 'POST:/api/v1/sales/*/purchase', 'SLIDING_WINDOW', 10, 60, NULL, NULL,
'USER', TRUE, NOW(), NOW()),
  (NULL, 'POST:/api/v1/sales/*/purchase', 'TOKEN_BUCKET',  100, 60,  200, 2.0,
'VIP',  TRUE, NOW(), NOW()),
  (NULL, 'GET:/api/v1/products*',          'FIXED_WINDOW',  60, 60, NULL, NULL,
'USER', TRUE, NOW(), NOW());
```

Database Design Decisions

Decision	Choice	Rationale
Soft deletes	deleted_at DATETIME NULL	Never hard-delete users or orders. Legal and audit requirements. Filter with WHERE deleted_at IS NULL.
UUIDs vs auto-increment	Both: internal BIGINT PK + UUID for external APIs	BIGINT PK for JOIN performance. UUID exposed in API so internal IDs are never leaked.
Idempotency key in orders	UNIQUE constraint on idempotency_key column	Second-layer safety net. Even if Redis idempotency layer fails, MySQL UNIQUE prevents duplicate order insert.
final_count in sale_events	Nullable column, populated post-sale	Reconciliation job reads final Redis inventory count and writes here. Proves Redis and MySQL agree.
DECIMAL for prices	DECIMAL(12,2) not FLOAT	Never use floating point for money. IEEE 754 rounding errors corrupt financial calculations.

Redis Key Design

06 Redis Key Design — Complete Reference

This section is the core learning material. Every Redis key is documented with its data structure, TTL, memory cost, and the algorithm that uses it.

Complete Key Registry

Redis Key	Structure + TTL	Purpose & Commands
rate:fw:{endpoint}:{userId}	STRING (integer) · TTL = window_seconds	Fixed window counter. INCR → if result==1 EXPIRE. O(1) time + O(1) memory.
rate:sw:{endpoint}:{userId}	SORTED SET · TTL = 2x window_seconds	Sliding window log. ZADD timestamp, ZREMRANGEBYSCORE to clean old, ZCARD to count. O(log N) per request.
rate:tb:{endpoint}:{userId}	HASH (tokens, last_refill) · No TTL	Token bucket. Lua script reads + calculates refill since last_refill + decrements atomically.
idem:{idempotency_key}	STRING (JSON response or PROCESSING) · 86400s TTL	Idempotency state. SETNX for initial set. GET to check. SET with full response when DONE.
inventory:{saleId}	STRING (integer) · TTL = sale_duration + buffer	Current stock. DECR for purchase. GETDEL at sale end for reconciliation.
user_purchases:{saleId}:{userId}	HASH (field: quantity) · TTL = sale_duration + 1h	Per-user purchase count within sale. HINCRBY on purchase. HGET to check limit.
sale_meta:{saleId}	HASH (startTime, endTime, maxPerUser, price) · 30s TTL	Cached sale metadata. Avoids MySQL hit on every purchase. Cache-aside pattern.
orders:queue	LIST (FIFO) · No TTL	Async order persistence queue. LPUSH by purchase service. BRPOP by OrderPersistenceJob.
stock:updates:{saleId}	Pub/Sub channel · No TTL	Real-time inventory broadcast. PUBLISH after every DECR. SSE endpoint subscribes.

Redis Key	Structure + TTL	Purpose & Commands
session:{jwtToken}	STRING (userId) · 86400s TTL	JWT denylist on logout. SET with user-id on login, DEL on logout.

Algorithm 1: Fixed Window Counter

Best for: simple protection, very low Redis memory. Bug: double traffic allowed at window boundary (last second of window N + first second of window N+1).

```
// FixedWindowStrategy.java
public RateLimitResult checkLimit(String userId, String endpoint, RateLimitConfig config) {
    String key = RedisKeyBuilder.fixedWindow(endpoint, userId);
    Long count = stringRedisTemplate.opsForValue().increment(key);
    if (count == 1) {
        // First request in window — set expiry
        stringRedisTemplate.expire(key, config.getWindowSeconds(), TimeUnit.SECONDS);
    }
    long remaining = Math.max(0, config.getRequestsLimit() - count);
    if (count > config.getRequestsLimit()) {
        return RateLimitResult.blocked(remaining, config.getWindowSeconds());
    }
    return RateLimitResult.allowed(remaining);
}

// Memory: one STRING per user per endpoint. ~64 bytes per key.
```

Algorithm 2: Sliding Window Log

Best for: precise rate limiting with no boundary bug. Cost: O(N) memory — stores every request timestamp in sorted set. Prune with ZREMRANGEBYSCORE.

```
// SlidingWindowStrategy.java
public RateLimitResult checkLimit(String userId, String endpoint, RateLimitConfig config) {
    String key = RedisKeyBuilder.slidingWindow(endpoint, userId);
    long now = System.currentTimeMillis();
    long windowStart = now - (config.getWindowSeconds() * 1000L);

    // Pipeline: remove old, add current, count, set TTL
    List<Object> results = stringRedisTemplate.executePipelined((RedisCallback<Object>) conn -> {
        conn.zRemRangeByScore(key.getBytes(), 0, windowStart);  // remove expired entries
        conn.zAdd(key.getBytes(), now, (now + "").getBytes());  // add current
        conn.zCard(key.getBytes());                              // count
        conn.expire(key.getBytes(), config.getWindowSeconds() * 2L);  // TTL = 2x window
        return null;
    });

    Long count = (Long) results.get(2);
    long remaining = Math.max(0, config.getRequestsLimit() - count);
    if (count > config.getRequestsLimit()) {
        return RateLimitResult.blocked(remaining, config.getWindowSeconds());
    }
    return RateLimitResult.allowed(remaining);
}

// Memory: O(requests_limit) per user per window. 1000 req/min = ~80KB per active user.
```

Algorithm 3: Token Bucket (Lua — Production Grade)

Best for: **handling burst traffic gracefully. Allows short bursts above steady rate. The only algorithm that won't reject a VIP user who sends 5 requests in 100ms after being idle.**

```
-- token_bucket.lua  (loaded at startup, called via EVALSHA)
local key          = KEYS[1]
local capacity     = tonumber(ARGV[1])   -- max tokens (burst_capacity)
local refill_rate  = tonumber(ARGV[2])   -- tokens added per second
local now          = tonumber(ARGV[3])   -- current epoch milliseconds

local data = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens      = tonumber(data[1]) or capacity
local last_refill = tonumber(data[2]) or now

-- Calculate tokens to add since last call
local elapsed = (now - last_refill) / 1000.0   -- convert to seconds
local refilled = math.min(capacity, tokens + elapsed * refill_rate)

if refilled < 1 then
    return {0, math.floor(refilled * 1000)}   -- blocked: {0, ms_until_next_token}
end

-- Consume one token
local new_tokens = refilled - 1
redis.call('HMSET', key, 'tokens', new_tokens, 'last_refill', now)
return {1, math.floor(new_tokens)}            -- allowed: {1, remaining_tokens}
```

Flash Sale Lua Script — The Core of Zero Oversell

Why Lua? Redis is single-threaded. A Lua script executes atomically — no other command can execute between the check and the decrement. Without Lua, two threads could both read inventory=1, both decide to proceed, and both decrement — resulting in inventory=-1 and two orders for the last unit.

```
-- flash_sale_purchase.lua
local inventory_key      = KEYS[1]   -- inventory:{saleId}
```

```
local user_purchase_key  = KEYS[2]   -- user_purchases:{saleId}:{userId}
local quantity           = tonumber(ARGV[1])   -- units to purchase (usually 1)
local max_per_user       = tonumber(ARGV[2])

-- Check current inventory
local current = tonumber(redis.call('GET', inventory_key))
if current == nil or current < quantity then
    return {-1, 0}   -- SOLD_OUT
end

-- Check per-user limit
local already_purchased = tonumber(redis.call('HGET', user_purchase_key, 'qty')) or 0
if already_purchased + quantity > max_per_user then
    return {-2, already_purchased}   -- LIMIT_EXCEEDED
end

-- Both checks passed — atomically decrement and record
local new_inventory = redis.call('DECRBY', inventory_key, quantity)
redis.call('HINCRBY', user_purchase_key, 'qty', quantity)

return {new_inventory, already_purchased + quantity}  -- SUCCESS: {new_stock, user_total}

-- Return codes: -1 = SOLD_OUT, -2 = LIMIT_EXCEEDED, >= 0 = new inventory count
```

Idempotency State Machine

```
// IdempotencyStore.java
public enum IdempotencyState { ABSENT, PROCESSING, DONE }

public boolean tryAcquire(String key) {
    // SETNX = SET if Not eXists — atomic, returns true only once
```

```
    Boolean acquired = stringRedisTemplate.opsForValue()
        .setIfAbsent(idemKey(key), "PROCESSING", Duration.ofHours(24));
    return Boolean.TRUE.equals(acquired);
}

public void storeResponse(String key, String responseBody) {
    // Store full response JSON — replaces PROCESSING
    stringRedisTemplate.opsForValue()
        .set(idemKey(key), "DONE:" + responseBody, Duration.ofHours(24));
}

public Optional<String> getStoredResponse(String key) {
    String value = stringRedisTemplate.opsForValue().get(idemKey(key));
    if (value == null) return Optional.empty();
    if (value.equals("PROCESSING")) return Optional.of("__PROCESSING__");
    return Optional.of(value.substring(5)); // strip "DONE:" prefix
}

// State transitions:
// ABSENT --[tryAcquire]--> PROCESSING --[storeResponse]--> DONE
// Duplicate request hits PROCESSING → 202 Accepted
// Duplicate request hits DONE → 200 with cached response body
```

API Design

07 API Design — Complete Specification

Auth Endpoints

Endpoint	Request	Response / Notes
POST /api/v1/auth/login	{ email, password }	{ accessToken (15min JWT), refreshToken (7d), userId, role }
POST /api/v1/auth/refresh	{ refreshToken }	{ accessToken (new 15min) } — refresh token rotation on every use
POST /api/v1/auth/logout	Bearer token in header	DEL session:{token} from Redis. Immediate invalidation.

Flash Sale — User Endpoints

Endpoint	Method + Auth	Request / Response
GET /api/v1/sales	PUBLIC	List active sales. Returns: [{ id, productName, salePrice, startTime, endTime }]
GET /api/v1/sales/{saleId}	PUBLIC	Single sale detail. Includes currentInventory from Redis (cached 2s TTL).
POST /api/v1/sales/{saleId}/purchase	USER · X-Idempotency-Key required	Body: { quantity: 1 }. Returns 200 { orderId, orderUuid, remainingStock } or 409 SOLD_OUT or 403 LIMIT_EXCEEDED. Rate limit: 10/min sliding window.
GET /api/v1/sales/{saleId}/inventory/stream	PUBLIC · SSE	Server-Sent Events stream. data: {remaining: 42}. Reconnect handled by browser EventSource.
GET /api/v1/orders/{orderUuid}	USER (own orders only)	Order confirmation details.

Admin Endpoints

Endpoint	Method	Purpose
POST /api/v1/admin/sales	ADMIN	Create sale event. Body: { productId, totalInventory, maxPerUser, salePrice, startTime, endTime }
POST /api/v1/admin/sales/{id}/activate	ADMIN	Immediately load inventory into Redis. SET inventory:{id} with TTL. Transitions status to ACTIVE.
GET /api/v1/admin/sales/{id}/stats	ADMIN	{ totalInventory, soldCount (from Redis), ordersInQueue (LLEN), mysqlOrderCount }
PUT /api/v1/admin/rate-limits/{id}	ADMIN	Update rate limit config. Hot-reloadable: RateLimitConfigCache @RefreshScope clears on update.
POST /api/v1/admin/sales/{id}/end	ADMIN	Manual sale termination. Triggers reconciliation: Redis inventory → MySQL final_count.

Standard Error Response Envelope

```
// All error responses follow this structure
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",       // machine-readable, used by client
    "message": "Too many requests",
    "details": {
      "limit": 10,
      "window": "60s",
      "retryAfter": 43                   // seconds until next allowed request
```

```
    }
  },
  "requestId": "b3a9f1c2-...",          // correlates with server logs
  "timestamp": "2025-06-21T10:30:00Z"
}

// HTTP status codes used:
// 200 — success
// 202 — idempotent request still processing
// 400 — validation error
// 401 — not authenticated
// 403 — limit exceeded (user bought max units)
// 404 — resource not found
// 409 — sold out / conflict
// 429 — rate limit exceeded
// 500 — unexpected server error (never expose stack trace)
```

Scaling Strategy

08 Scaling Strategy — 1K to 10M Users

Stage 1: 0–10,000 Users — Single Instance

Configuration
Single Spring Boot instance. Docker Compose local. Redis standalone. MySQL single node. This is your learning setup.

Layer	Configuration	Limit Before Upgrade
Spring Boot	Single instance, 8 threads (default)	~500 concurrent users before CPU saturation
Redis	Standalone, single node, AOF persistence	~100K ops/sec — not the bottleneck at this stage
MySQL	Single instance, HikariCP pool size 20	~2K QPS on indexed queries
Rate Limiter	All algorithms work on single Redis	No distribution concerns yet

Stage 2: 10,000–100,000 Users — Horizontal Scale

Layer	Change	Why Now
Spring Boot	Add second instance behind Nginx load balancer	Single instance CPU becomes bottleneck at ~5K concurrent users
Redis	Switch to Redis Sentinel (1 master, 2 replicas)	Rate limit state must be shared across Spring Boot instances — in-memory state breaks multi-instance
MySQL	Add read replica for product/sale GET queries	Write traffic stays on primary. Read replicas serve browse traffic.
Rate Limiter	No algorithm change needed	Redis is already the shared state store — multi-instance works automatically

Stage 3: 100,000–1,000,000 Users — Cluster Mode

Layer	Change	Why Now
Spring Boot	Kubernetes deployment, HPA on CPU/ memory	Pod autoscaling handles flash sale spikes. Scale to 0 between sales.
Redis	Redis Cluster (3 shards, 3 replicas = 6 nodes)	Single Redis node saturates at ~1M ops/sec. Cluster shards by key hash.
Redis (rate limiting)	Lua scripts MUST use only KEYS on same slot	Redis Cluster constraint: Lua cannot cross key slots. Use hash tags: {userId}:rate:sw to force same slot.
MySQL	Vertical scale + connection pooling (PgBouncer-equivalent)	Connection count limits before DB itself. ProxySQL or connection pool layer needed.
Orders queue	Evaluate Kafka migration	Redis List BRPOP is single consumer. If order volume exceeds 10K/min, Kafka consumer groups provide parallelism.

Stage 4: 1M–10M Users — Sharded Architecture

Layer	Change	Why Now
Redis	Per-sale Redis instance during sale window	A single Flipkart Big Billion Day sale generates 500K ops/sec on inventory:{saleId} alone. Isolated Redis per sale event.
MySQL	Shard orders table by user_id % N	Orders table exceeds 100M rows. Single-node queries degrade. Vitess or manual sharding.

Layer	Change	Why Now
Rate Limiter	Approximate counting acceptable	At 10M users, Redis sorted set memory for sliding window (80KB per user) = 800GB. Switch to Fixed Window or probabilistic counting (Count-Min Sketch).
Idempotency	Distributed idempotency with Cassandra	24h Redis retention for 10M users × 5 keys = 50M keys × ~200 bytes = 10GB. Redis handles this, but add Cassandra for long-term idempotency audit trail.

Best Practices & Testing

09 Engineering Best Practices

Redis Key Naming Convention

```
// Pattern: {feature}:{algorithm_or_type}:{identifier}:{sub_identifier}
// All keys centralised in RedisKeyBuilder.java

public class RedisKeyBuilder {
    private static final String SEP = ":";

    // Rate limiting
    public static String fixedWindow(String endpoint, String userId) {
        return "rate:fw:" + encode(endpoint) + SEP + userId;
    }
    public static String slidingWindow(String endpoint, String userId) {
        return "rate:sw:" + encode(endpoint) + SEP + userId;
    }
    public static String tokenBucket(String endpoint, String userId) {
        return "rate:tb:" + encode(endpoint) + SEP + userId;
    }

    // Flash sale
    public static String inventory(Long saleId) {
        return "inventory:" + saleId;
    }
    public static String userPurchases(Long saleId, Long userId) {
        return "user_purchases:" + saleId + SEP + userId;
    }

    // Idempotency
    public static String idempotency(String key) {
        return "idem:" + key;
    }

    // NEVER hardcode Redis key strings outside this class
```

```
}
```

Testing Strategy

Test Type	Tool	What to Test
Unit tests	JUnit 5 + Mockito	Each rate limiting algorithm in isolation. Mock RedisTemplate. Test boundary conditions: exactly at limit, at limit+1, window rollover.
Integration tests	@SpringBootTest + Testcontainers	Full request through filters → service → Redis (real Redis container) → MySQL. Assert Redis key state after purchase.
Concurrency tests	CountDownLatch + ExecutorService	500 threads hit purchase simultaneously. Assert: orders count == inventory count, no inventory < 0.
Load tests	k6 (Docker)	Real HTTP traffic, 500 VUs, 30 seconds. Assert p95 < 200ms, error rate < 0.1%.
Algorithm comparison	Custom benchmark	Run 10K requests against fixed, sliding, token bucket. Compare memory (MEMORY USAGE key) and latency (INFO latency).

Concurrency Test — The Most Important Test

```
// FlashSaleServiceTest.java — proves zero oversell
@Test
void testConcurrentPurchases_noOversell() throws InterruptedException {
    int INVENTORY = 100;
```

```
    int THREADS   = 500;   // 5x oversubscribed

    // Load inventory into Redis
    redisTemplate.opsForValue().set("inventory:" + SALE_ID,
String.valueOf(INVENTORY));

    CountDownLatch startGate = new CountDownLatch(1);
    CountDownLatch endGate   = new CountDownLatch(THREADS);
    AtomicInteger successCount = new AtomicInteger(0);
    AtomicInteger soldOutCount = new AtomicInteger(0);

    for (int i = 0; i < THREADS; i++) {
        final long userId = i;
        executor.submit(() -> {
            try {
                startGate.await();  // all threads start simultaneously
                PurchaseResult result = flashSaleService.purchase(SALE_ID, userId,
1);
                if (result.isSuccess()) successCount.incrementAndGet();
                else soldOutCount.incrementAndGet();
            } catch (Exception e) {
                soldOutCount.incrementAndGet();
            } finally {
                endGate.countDown();
            }
        });
    }

    startGate.countDown();   // release all 500 threads at once
    endGate.await(30, TimeUnit.SECONDS);

    // Critical assertions
    assertEquals(INVENTORY, successCount.get(),
        "Exactly " + INVENTORY + " purchases should succeed — no more, no less");
    assertEquals(THREADS - INVENTORY, soldOutCount.get());
```

```
    // Verify Redis state
    String remaining = redisTemplate.opsForValue().get("inventory:" + SALE_ID);
    assertEquals("0", remaining, "Inventory must be exactly 0, not negative");
}
```

Algorithm Comparison Cheat Sheet

Property	Fixed Window	Sliding Window	Token Bucket
Memory per user	O(1) — 1 key	O(N) — N timestamps	O(1) — 1 hash
Boundary bug	Yes — 2x traffic at edge	No	No
Burst handling	Hard cutoff	Hard cutoff	Allows bursts up to capacity
Redis operations	2 (INCR + EXPIRE)	4 pipelined	1 Lua (EVALSHA)
Implementation complexity	Trivial	Medium	Medium (Lua required)
Best use case	High-volume public endpoints	Purchase endpoints	VIP/premium APIs
Interview answer	Show the boundary bug first	Show this is the fix	Show Lua atomicity

Interview Talking Points

Rate Limiter

"I implemented all three algorithms and benchmarked them. Fixed window has a known boundary vulnerability — I can show you the exact scenario where it allows 2x traffic. Sliding window fixes this but costs O(N) memory per user. Token bucket via Redis Lua is the production choice because the Lua script guarantees atomicity and it allows VIP users short burst traffic."

Flash Sale

"The core problem is that MySQL row locks serialize under high concurrency — I measured 400ms latency at 500 concurrent requests. Redis DECR is atomic at the server level and doesn't block. My Lua script does a check-and-decrement atomically, so even 5,000 concurrent threads can't oversell. I have a concurrency test with 500 threads proving zero oversell."

Idempotency

"Double-click on Buy Now sends two requests. Without idempotency, a user gets two orders. I use SETNX to claim a key with PROCESSING state — only one request wins the claim. When it completes, I store the full response body in Redis. Any retry gets the cached response, never a second DB write. This is exactly how Stripe and Razorpay implement it."